

Phase 3 – Testing Our Game

This report walks through our testing phase, focusing on how we validated the game's core components through unit and integration tests.

Our goal for this phase was to use JUnit to verify the correctness of individual features on their own as well as the interactions between key components of our system. We wrote our tests using the JUnit framework, and they are fully automated through Maven. This report goes in-depth into the features and interactions tested, discusses the quality measures taken, presents code coverage results, and summarizes the findings and improvements made to the production code during the testing process.

Unit Tests

We spread the unit tests for the map creation subsystem across `Tile_TypeTest`, `Map_TileTest`, and `Map_BuilderTest`, and each targets a specific piece of logic in isolation. `Tile_TypeTest` contains seven tests that verify the `isSolid()` method on every value of the `Tile_Type` enum, confirming that walkable tiles like `Floor` return false and all blocking tiles, such as `Wall`, `Shelf`, `Tech`, `Meat`, `Produce`, and `Blocked`, return true. In `Map_TileTest`, the unit-level tests focus on the constructor and the `isSolid()` method of `Map_Tile`. For each valid character input, a test asserts that the correct `Tile_Type` is assigned and that solidity is reported correctly, and an additional test confirms that passing an unrecognized character leaves the type as null. In `Map_BuilderTest`, the unit tests cover the dimension methods, verifying that the map is 20 rows by 27 columns with a tile size of 30 pixels, as well as the `findTile()` method, which is tested for all seven tile types to confirm it returns valid coordinates, and for an unknown character to confirm it returns null. The four boundary tests for `isSolid()` are also unit tests, verifying that out-of-bounds pixel coordinates in all four directions are correctly treated as solid without accessing any tile data.

Our unit tests for the player and enemy subsystems are covered in `PlayerUnitTest` and `SecurityGuardUnitTest`. `PlayerUnitTest` verifies that a newly constructed player starts alive, that `setDead()` correctly transitions the player of the game to a non-alive state, that the constructor correctly sets the initial position, and that animation frames default to `FRAME_DOWN` and update correctly. `SecurityGuardUnitTest` verifies that the guard's constructor correctly sets position, that `chasePlayer()` transitions the action state to `Chase`, that the default enemy size is 30, and that `ChaseState` is correctly initialized and updated through `createChaseState()` and `resetChaseState()`.

Our unit tests for game state management are covered in `GamePanelStateTest`, `GameRulesHelperTest`, and `GameStateResetHelperTest`. `GamePanelStateTest` verifies that all

four state flags (started, paused, over, and won) initialize to false and can each be set independently without affecting the others. GameRulesHelperTest exhaustively covers the win condition logic, testing both outcomes of isPlayerAtExit() and all four paths through hasWon(). GameStateResetHelperTest verifies that a full reset correctly restores the player's position, all game flags, animation frames, security guard position, and timing and enemy state values to their defaults.

Our unit tests for stationary entities are covered in Entity_Stationary_Testing. This class verifies getter and setter correctness for Item_Main, Item_Bonus, and Item_Penalty, confirms that collectIfTouched() returns the correct value and sets the collected flag on overlap, and returns zero when out of range. The areAllCollected() method is tested across all three scenarios, and a mixed scoring test confirms the correct combined total across all three item types. Spawn limit tests verify that each spawner respects its defined item limit when interacting with Map_Builder.

List of Unit Test Files

Test Method	Description
PlayerUnitTest.java	Tests core behaviour of the Player class: construction, lifecycle state, position initialization, and animation frame management.
SecurityGuardUnitTest.java	Tests the SecurityGuard class in isolation: position initialization, action state transitions, size defaults, and chase state management.
GamePanelStateTest.java	Tests the state flag management of GamePanel: game states (started, paused, over, won) are correctly initialized and updated.
GameRulesHelperTest.java	Tests static win-condition logic in GameRulesHelper: exit detection and the combination of conditions required to trigger a game win.
GameStateResetHelperTest.java	Tests GameStateResetHelper, verifying that a full game reset correctly restores player position, game flags, animation frames, enemy state, and timing values.

Tile_TypeTest.java	Tests the Tile_Type enum, verifying that each tile type correctly reports whether it is solid (impassable) or not.
Map_TileTest.java	Tests individual Map_Tile objects: each tile character maps to the correct type, solid flag, and renders pixels when drawn.
Entity_Stationary_Testing.java	Tests stationary entity classes (Item_Main, Item_Bonus, Item_Penalty) and their spawning logic. Verifies getter and setter correctness, spawn limits for each item type, collection detection and scoring, the collected flag, and the all-collected check used to determine win eligibility.

Integration Tests

The integration tests for the map creation subsystem focus on how Map_Builder and Map_Tile work together. For example, the draw() test in Map_BuilderTest calls map_builder.draw(graphics) and then checks that pixels are actually drawn on the image. Because of this, a failure in the test could point to an issue in either class or in how they interact. Similarly, tests like testRandomWall and testRandomFloor also have an integration aspect. The findTile() method creates a new Map_Tile as a reference and compares it to tiles in the map, so these tests rely on both correct tile construction and proper storage/retrieval within Map_Builder. The draw() tests in Map_TileTest also carry some integration weight, since they depend on image resources being loaded successfully by Map_Tile's static initializer. If that initialization fails, the draw tests would fail regardless of whether the actual drawing logic is correct.

The integration tests for the player and enemy subsystems are covered in PlayerEnemyIntegrationTest. This class tests the interaction between Player, SecurityGuard, and Map_Builder working together. It verifies that collision detection correctly identifies overlap between the player and guard, that the guard's pathfinding moves it closer to the player each update tick, that both the player and guard are blocked by solid tiles from Map_Builder, and that the catch condition and chase state target clearing behave correctly when the guard reaches its waypoint.

The integration tests for the AI subsystem are covered in PathfinderIntegrationTest. This class tests the interaction between the Pathfinder module and Map_Builder, since the Pathfinder depends on the map's solid-tile data to compute valid routes. It verifies that a non-null next step

is returned when a clear path exists, and that the returned step is exactly one tile away from the starting position, confirming that the two components cooperate correctly.

The integration tests for the game panel are covered in `GamePanel_Testing`. This class tests `GamePanel` against the keyboard input system, game state flags, and the rendering pipeline together. It verifies that key presses for starting, pausing, and resetting the game produce the correct state transitions, that movement input is safely ignored in invalid states such as before start, while paused, and on game over or win, and that `paintComponent()` runs without throwing exceptions across all screen states.

List of Integration Test Files

Test Method	Description
<code>PathfinderIntegrationTest.java</code>	Tests the interaction between the Pathfinder AI module and <code>Map_Builder</code> . The pathfinder relies on the map's solid-tile data to compute a valid route.
<code>Map_BuilderTest.java</code>	Tests <code>Map_Builder</code> and its integration with <code>Map_Tile</code> and <code>Tile_Type</code> . Verifies correct map construction, tile type resolution, out-of-bounds handling, and rendering.
<code>PlayerEnemyIntegrationTest.java</code>	Primary integration test suite testing interactions between <code>Player</code> , <code>SecurityGuard</code> , and <code>Map_Builder</code> : collision detection, movement blocking, guard chase behaviour, and catch conditions.
<code>GamePanel_Testing</code>	Tests <code>GamePanel</code> and its integration with the keyboard input system, game state flags, and rendering pipeline. Verifies correct key press handling, safe behaviour during invalid movement states, and exception-free rendering across all screen states.

Test Quality

Several measures were taken to ensure the quality of the test suite. Test methods are named descriptively using a subject-action-outcome convention (e.g., `guard_movesCloserToPlayer_afterOneUpdate`, `hasWon_returnsFalse_whenGameAlreadyWon`), making the intent and expected behaviour of each test immediately clear without needing to read the implementation. Nearly all assertions include a failure message that explains what went wrong, improving the diagnostic value of failed runs. Shared setup logic is extracted into `@BeforeEach` methods, keeping individual tests concise and ensuring each test starts from a clean, consistent state. Both positive and negative cases are covered throughout — for example, `GameRulesHelperTest` tests all four paths through the win condition logic rather than only the success case. Boundary conditions are explicitly addressed in `Map_BuilderTest`, which tests all four out-of-bounds directions to verify that the map handles edge access safely. Unit tests construct their own objects directly rather than relying on shared or global state, keeping each test independent and repeatable. Finally, rendering tests in `GamePanel_Testing` use a `BufferedImage` as a mock graphics context instead of requiring a real display, allowing UI code to be tested reliably in a headless environment.

Code Coverage

Code coverage was measured using IntelliJ's built-in coverage tool. Overall, the test suite achieved 90% class coverage, 90% method coverage, 92% line coverage, and 80% branch coverage.

In the `mapCreation` package, we reached 100% class and method coverage, along with 97% line coverage and 75% branch coverage. This was achieved through unit and integration tests in `Map_TileTest`, `Map_BuilderTest`, and `Tile_TypeTest`, which exercised all tile types, boundary conditions, and draw methods. The `enemy` package reached 100% class coverage, 93% method coverage, 96% line coverage, and 72% branch coverage. The `SecurityGuard`'s movement, collision, and chase behaviour were tested using both `SecurityGuardUnitTest` and `PlayerEnemyIntegrationTest`. The unit tests focused on initial position, chase behaviour, enemy size, and state initialization/reset, while the integration tests verified that the guard moves toward the player, is blocked by solid tiles, catches the player correctly, and clears its waypoint after reaching a target. The `ai` package achieved 95% line coverage and 80% branch coverage for its pathfinding class. `PathfinderIntegrationTest` ensured that the pathfinder returns a valid next step when a path exists and that the step is exactly one tile away from the starting position. `GamePanel` reached 95% method coverage and 93% line coverage. This was achieved through `GamePanel_Testing`, which simulates key presses to cover start, pause, reset, and movement logic, and `GamePanelStateTest`, which verifies game state flags using test-only setters. The `Player` class reached 100% line coverage through `PlayerUnitTest`, which checks initial position,

alive state on construction, death state after `setDead()`, default frame, and frame updates across all directions. `PlayerMovementHelper` achieved 100% line coverage and 85% branch coverage through tests in `PlayerEnemyIntegrationTest`, confirming that the player is blocked by solid tiles and moves correctly on open floor. Item and spawning logic were tested in `Entity_Stationary_Testing`. These tests covered getters, setters, spawn limits for all item types, collection range, return values, the collected flag, the `areAllCollected()` method, and scoring across main, bonus, and penalty items. Finally, `GameRulesHelperTest` and `GameStateResetHelperTest` covered win conditions and reset behaviour. These verified all flag combinations for `hasWon()`, exit overlap detection, and correct default values for player position, game state flags, timing, and enemy state after a reset.

Findings (bug findings)

While writing and running our test suites revealed several bugs, code smells, and areas for improvement to us all across the codebase. For the map creation subsystem, the `Map_Builder` class, we refactored it by extracting the map construction logic out of the constructor and into a dedicated method, which improved clarity significantly. We also created a `findTile()` method, which was also added to support integration testing between `Map_Builder` and `Map_Tile`.

The testing process also exposed a few broader design issues to us. In several cases, the tests showed that some classes were either doing too much work or were too tightly coupled. This made them much harder to isolate and verify, and was a challenge for us at the start. This led to us making small but meaningful improvements in code structure, especially around separating setup logic from core behaviour and making the codebase more test-friendly. In addition, running the tests helped us uncover missing or weak assertions in some areas, which improved the reliability of the test suite itself.

Overall, this phase reinforced the value of writing our tests alongside production code rather than after the fact. Several assumptions about how components behaved turned out to be incorrect once assertions were in place, and the process of making code testable naturally pushed our team toward better separation of concerns and cleaner interfaces between all of our classes. Code coverage results also helped identify areas of the codebase that had no test coverage at all.